AI Wikipedia

Privacy-First AI Encyclopedia with Verification

Version 1.1.0 — March 2026

--- ## Overview AI Wikipedia is a privacy-first application that generates articles using local Ollama LLM and verifies claims against 20+ authoritative sources including Lex.dk (Den Store Danske).

AI Article Generation
Create Wikipedia-style articles using local Ollama LLM

AI Chat Interface
CLI-style terminal for asking questions directly

Multi-Source Verification
Verify claims against 20+ authoritative sources

Compressed Storage
LZ4/Zstd compression for Wikipedia dumps

--- ## Features - **AI Article Generation** — Create articles using local Ollama LLM - **AI Chat Interface** — CLI-style terminal for asking questions directly (always visible) - **Multi-Source Verification** — Verify claims against 20+ authoritative sources - **Lex.dk (Den Store Danske)** — Denmark's National Encyclopedia (most trusted) - **Wikipedia & WikiData** — International encyclopedia and structured data - **20+ Danish & International APIs** — Statistics, legal, business, and more - **Compressed Storage** — LZ4/Zstd compression for Wikipedia dumps (16GB → ~5-6GB) - **Import from Wikipedia** — Import real Wikipedia articles - **Portable & Syncable** — Export/import JSON for syncing across devices - **Dark Mode** — Built-in theme toggle --- ## Quick Start ### Prerequisites 1. **Ollama** — Install from [ollama.ai] (https://ollama.ai) ```bash # Install Ollama curl -fsSL https://ollama.ai/install.sh | sh # Start Ollama service ollama serve # Pull default model (llama3.2:1b) ollama pull llama3.2:1b ``` 2. **Node.js** — v18+ required ### Installation ```bash # Clone or copy the project cd /path/to/ai-wikipedia # Install dependencies npm install # Run in development mode npm run dev ``` Open http://localhost:1420 in your browser. --- ## Usage ### AI Chat (Always Visible) The AI CLI chat is always visible under the search bar: - Select model: llama3.2, mistral, phi3, codellama - Type

questions directly and press Send or Enter - Click "Save as Article" to save responses ### Generate Article with AI 1. Click "+ Generate with AI" 2. Enter a topic (e.g., "Quantum Computing") 3. Click Generate 4. Article is created using Ollama locally ### Verify Claims 1. Open any article 2. Click "Verify Claims" 3. Each claim is checked against enabled sources 4. Results show: ✓ Verified, ? Disputed, or − Unverified ### Enable/Disable Sources 1. Click "Sources" in the header 2. Toggle sources on/off 3. Add custom API endpoints 4. Changes persist in localStorage ### Import from Wikipedia 1. Click "Import Wikipedia" 2. Enter article titles (one per line) 3. Click "Import" 4. Articles are added with verified status ### Export / Sync Data 1. Click "Export / Sync" 2. Click "Export All" to download JSON 3. Save to USB or cloud storage 4. On target device, use "Import from JSON" ### Compressed Storage 1. Click "Storage" in the header 2. Choose compression: **LZ4** (fast) or **Zstd** (better ratio) 3. Click "Save to Compressed Storage" 4. View compression stats: original vs compressed size 5. Export compressed bundles for backup --- ## Verification Sources ### Primary Sources (Enabled by Default) | Source | Type | Description | |--------|------|-------------| | **Lex.dk** | Encyclopedia | Denmark's National Encyclopedia (Den Store Danske) | | **Wikipedia** | Encyclopedia | International encyclopedia | | **WikiData** | Structured Data | Structured knowledge base | ### Danish Public APIs | Source | Keywords | Description | |--------|----------|-------------| | Danmarks Statistik | statistics, population | Statistics Denmark API | | CVR.dk | company, virksomhed | Danish Business Register | | Retsinformation | law, legal, folketing | Danish legal documents | | Nationalbanken | currency, valuta | Exchange rates | | Dataforsyningen | geo, address | Geospatial data | | DMI Vejr | weather, vejr | Weather data | | MotorAPI | vehicle, bil | Vehicle registration | | SSI COVID-19 | covid, coronavirus | COVID-19 surveillance data | | Folketinget | parliament, politics | Parliamentary documents | | DSB Feeds | train, tog | Train schedules | | Energi Data Service | energy, solar, wind | Energy production data | | elprisenligenu.dk | electricity, strom | Electricity prices | ### International Sources | Source | Keywords | Description | |--------|----------|-------------| | Eurostat | eu, european | EU Statistics | | IMF Data | imf, global economy | International Monetary Fund | | OECD Data | oecd | Economic cooperation data | | SEC EDGAR | sec, edgar | US company filings | | EUR-Lex | eu law | EU legislation | | EconPapers | economics, research | Economics working papers | --- ## Compression ### Expected Ratios | Algorithm | Ratio | Speed | CPU Impact | |-----------|-------|-------|------------| | LZ4 | ~40% | 2GB/s | Very Low | | Zstd | ~35% | 500MB/s | Low-Med | **For 16GB Wikipedia:** ~5-6GB with LZ4, ~4-5GB with Zstd --- ## Privacy - All AI processing uses local Ollama — no data sent to external AI services - Verification uses public APIs (read-only) - Data stored locally on your machine - Export files are unencrypted JSON — keep them secure --- ## Tech Stack - **Frontend:** React 18 + TypeScript + TailwindCSS - **Build Tool:** Vite 5.4 - **Desktop:** Tauri 2.0 (Rust) - **Compression:** LZ4 + Zstd (Rust) - **AI:** Ollama

(local LLM) - **Verification:** 20+ APIs (Wikipedia, WikiData, Lex.dk, Danish APIs) --- ## Project Structure ``` ai-wikipedia/ ├── src/ │ ├── App.tsx # Main application (~1900 lines) │ ├── main.tsx # React entry point │ └── index.css # Tailwind styles ├── src-tauri/ # Tauri desktop app │ ├── src/ │ │ ├── lib.rs # Rust commands (compression) │ │ └── main.rs # Entry point │ ├── Cargo.toml # Rust dependencies │ └── tauri.conf.json # Tauri config ├── package.json ├── vite.config.ts ├── tailwind.config.js ├── SPEC.md # Feature specification ├── CHANGELOG.md # Version history └── README.md # This documentation ``` --- ## Troubleshooting ### "Ollama not available" ```bash # Start Ollama ollama serve # Check if running curl http://localhost:11434/api/tags # List available models ollama list ``` ### Build fails on Tauri ```bash # Update Rust rustup update # Clean and rebuild cd src-tauri cargo clean cargo update ``` --- ## License MIT License